

Assignment Cover Sheet / Declaration of Originality

Complete this form if/as directed by your unit coordinator, lecturer or the assignment specification.

Last name:	Perera	Student ID:	23601321
Other name(s):	Isini Ayansa		
Unit name:	Data Structures and Algorithms	Unit ID:	COMP1002
Lecturer / unit coordinator:	Ms Ying Li	Tutor:	Dr Umanga Pilapitiya
Date of submission:	02/06/2026	Which assignment?	(Leave blank if the unit has only one assignment.)

I declare that:

- The above information is complete and accurate.
- The work I am submitting is *entirely my own*, except where clearly indicated otherwise and correctly referenced.
- I have taken (and will continue to take) all reasonable steps to ensure my work is *not accessible* to any other students who may gain unfair advantage from it.
- I have *not previously submitted* this work for any other unit, whether at Curtin University or elsewhere, or for prior attempts at this unit, except where clearly indicated otherwise.

I understand that:

- Plagiarism and collusion are dishonest, and unfair to all other students.
- Detection of plagiarism and collusion may be done manually or by using tools (such as Turnitin).
- If I plagiarise or collude, I risk failing the unit with a grade of ANN ("Result Annulled due to Academic Misconduct"), which will remain permanently on my academic record. I also risk termination from my course and other penalties.
- Even with correct referencing, my submission will only be marked according to what I have done myself, specifically for this assessment. I cannot re-use the work of others, or my own previously submitted work, in order to fulfil the assessment requirements.

- It is my responsibility to ensure that my submission is complete, correct and not corrupted.

Signature: Isini

Date of signature: 02/06/2026

(By submitting this form, you indicate that you agree with all the above text.)

COMP1002
Final Assignment

ZIPRIDE DISPATCH SYSTEM: DATA STRUCTURES AND ALGORITHMS FOR EFFICIENT RIDE-HAILING

Name: Isini Ayansa Perera
Student ID: 23601321

Table of Contents

1. Introduction	6
2. Justification for the use of Data Structures choices	6
2.1 Graph	6
2.2 Hash Table.....	6
2.3 Heap	7
2.4 Sorting Algorithms	7
3. Module Interactions	7
3.1 Interconnectivity of modules	7
3.2 UML class Diagram	8
3.3 Data Flow Through the System	8
4. Time and Space Analysis	9
4.1 Module 1: Graph-Based Route Planning	9
4.2 Module 2: Hash-Based Passenger and Driver Lookup	9
4.3 Module 3: Heap-Based Pickup Scheduling	10
4.4 Module 4: Sorting Pickup Requests	10
5. Sample Outputs of Code	11
5.1 Module1:	12
5.1.1 Road network	12
5.1.2 Breadth First Search (BFS)	13
5.1.3 Depth First Search.....	13
5.1.4 Check for cycles	13
5.1.5 Shortest Path	14
5.2 Module2:	14
5.2.1 Add passenger	14
5.2.2 Search Passenger	14
5.2.3 Collision Handling	15
5.2.4 Add driver	15
5.2.5 Update driver status	15
5.2.6 Delete driver	16
5.3 Module3:	16
5.3.1 Create ride request	16
5.3.2 View priority queue	16
5.3.3 View highest priority	17
5.3.4 Dispatch next ride	17
5.4 Module4:	17

5.4.1	Merge sort execution	17
5.4.2	Quick sort execution	18
5.4.3	Comparison of Merge and Quick sort	18
6.	Module 4: Discussion	19
6.1	Merge Sort	19
6.2	Quick Sort	19
6.3	Comparison	19
7.	Conclusion	20
8.	References	20

1. Introduction

The ZipRide-Dispatch System was built to show how data structures and algorithms can help solve problems in transportation and scheduling. It has four parts:

- A Graph part for managing roads and finding routes
- A Hash Table part for storing and finding passenger and driver information
- A Heap part for prioritizing and scheduling rides
- A Sorting part for comparing how well Merge Sort and Quick Sort work, with ride data

By putting these parts the project shows how different data structures can be used to manage data, plan routes, schedule rides and analyze performance. The Zip Ride-Dispatch System uses a Graph to handle road networks and route calculations. The system also relies on a Hash Table for storage and retrieval of passenger and driver records. In addition, a heap is utilized for prioritizing and scheduling ride requests. Moreover, a Sorting module is included for analyzing the performance of Merge Sort and Quick Sort on ride request datasets. This report talks about the design choices how it was implemented its complexity, test results and overall evaluation of the Zip Ride-Dispatch System. The report also discusses the design decisions made for the zip Ride-Dispatch System. It explains the approach used to implement the system. The complexity of the Zip Ride-Dispatch System is also analyzed. The report presents the results of testing the Zip Ride-Dispatch System. Finally, it provides an evaluation of the Zip Ride-Sharing System. The Zip Ride-Dispatch System is evaluated.

2. Justification for the use of Data Structures choices

2.1 Graph

A graph data structure was chosen to depict the road network as it inherently represents locations and the roads linking them. In the ZipRide framework, every location is depicted as a vertex, and every road is shown as an edge with a corresponding travel time.

This framework supports the effective execution of graph algorithms like Breadth First Search (BFS), Depth First Search (DFS), cycle detection, and Dijkstra's shortest path algorithm. Employing a graph enables the system to execute route calculations and network analyses more efficiently than linear data structures.

2.2 Hash Table

A hash table was selected to handle passenger and driver records due to its efficient capabilities for insertion, searching, updating, and deletion operations. In the ZipRide system, every passenger and driver is recognized by a unique ID, which serves as the key in the hash table. Linear probing through open addressing was utilized to manage collisions and ensure quick record access. In contrast to linear searching in a list, a hash table provides much superior average-case efficiency, which makes it ideal for effectively handling a large volume of driver and passenger records.

2.3 Heap

A heap data structure was chosen to develop the ride scheduling system due to its effective management of ride requests based on their priority. In the ZipRide system, ride requests receive a priority score determined by the passenger's membership level and the anticipated pickup time. A max heap guarantees that the most important ride request is consistently positioned at the root, allowing for efficient access and dispatch. This method is preferable to relying on a standard list as it eliminates the need to continually search through every ride request to identify the next ride to schedule.

2.4 Sorting Algorithms

Sorting algorithms were utilized to examine and contrast various methods for arranging ride request data. Merge Sort and Quick Sort were chosen as both are popular comparison-based sorting algorithms known for their effective average-case performance. The algorithms underwent testing with datasets of varying sizes and configurations, such as random, almost sorted, and reversed data. Performance was assessed by calculating the execution time and counting the comparisons made. This analysis offers a look into the advantages and drawbacks of each algorithm in various scenarios and illustrates their practical use in the ZipRide system.

3. Module Interactions

3.1 Interconnectivity of modules

The four components of the ZipRide system are designed to function collectively as a cohesive ridesharing platform. The Graph module oversees the road network by keeping track of locations and roads, and algorithms like Dijkstra's algorithm are employed to determine the shortest travel time between locations. The Hash Table module oversees passenger and driver records, ensuring efficient storage and retrieval of user data. Upon the creation of a ride request, the system utilizes the passenger's pickup point along with the driver's existing location to calculate the estimated pickup time via the Graph module.

This data is subsequently integrated with the traveler's membership level to determine a priority score for the ride demand. The ride request is then placed into the Heap module, which organizes requests based on priority and guarantees that rides with higher priority are dispatched first. Ultimately, the Sorting module creates ride request datasets and implements Merge Sort and Quick Sort to evaluate sorting efficiency across various dataset scenarios. This integration allows the modules to jointly facilitate route management, record administration, ride planning, and performance assessment of algorithms within one unified system.

3.2 UML class Diagram

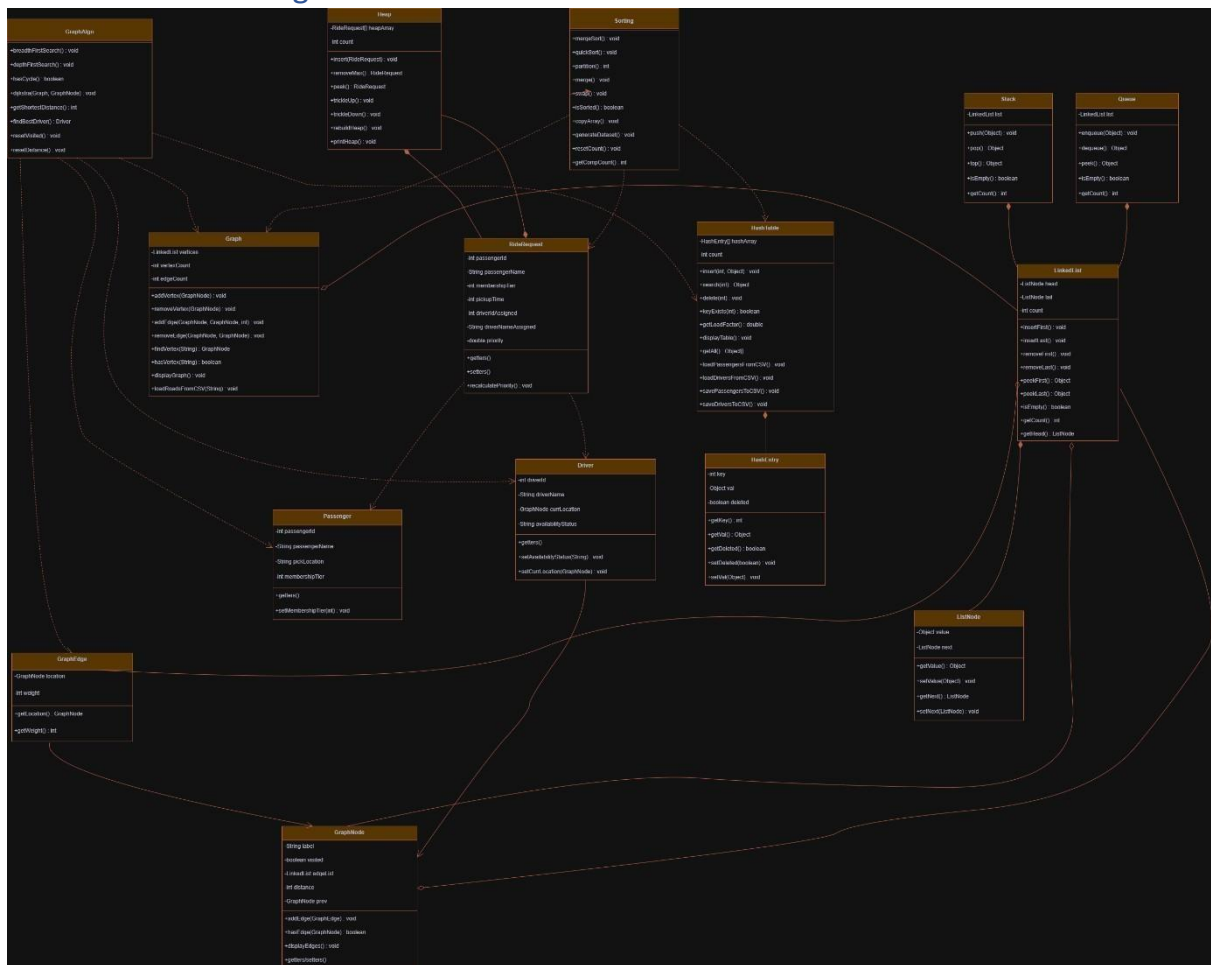


Figure 1 UML diagram for Zip ride dispatch system

3.3 Data Flow Through the System

The ZipRide system starts its operation when records of passengers and drivers are imported into the Hash Table module from CSV files. The Graph module maintains the road network through locations and roads, facilitating route computations between various points in the system. Upon a ride request by a passenger, the passenger record is fetched from the passenger hash table, and a free driver is chosen from the driver hash table. The Graph module subsequently computes the estimated pickup duration from the driver's present location to the passenger's pickup point utilizing Dijkstra's shortest path algorithm. The pickup time, along with the passenger's membership level, is utilized to determine the ride priority based on the given priority formula. The generated ride request is added to the Heap module, where requests are kept in order of priority. Ride requests with higher priority are sent out before those with lower priority. For performance evaluation, ride request datasets are created and handled by the Sorting module, which employs Merge Sort and Quick Sort to analyze execution time and comparison counts across various dataset conditions.

4. Time and Space Analysis

This section analyses the time and space complexity of the main data structures and algorithms implemented within the ZipRide Ride-Sharing System. Understanding these complexities helps evaluate the efficiency and scalability of each module.

4.1 Module 1: Graph-Based Route Planning

The Graph module stores locations as vertices and roads as weighted edges. Several graph algorithms are implemented to support route management and network analysis.

Algorithm	Purpose	Time Complexity	Space Complexity
Breadth First Search (BFS)	Input: source location (e.g. CBD). Output: all reachable locations grouped by level (Level 0, Level 1, ...).	$O(V+E)$	$O(V)$
Depth First Search (DFS)	Input: source location. Output: whether the graph contains a cycle; if found, show the locations involved	$O(V+E)$	$O(V+E)$
Cycle detection	Detect cyclic routes in the road network	$O(V+E)$	$O(V)$
Dijkstra- Shortest path	Determines the shortest path between two locations in the road network	$O(V^2)$	$O(V)$

Notation

- **V** = Number of vertices (locations)
- **E** = Number of edges (roads)

The Graph module models the transportation network using vertices and weighted edges. BFS and DFS are used for graph traversal and connectivity analysis, while cycle detection identifies loops within the network. Dijkstra's algorithm calculates the shortest path between locations and is used when estimating pickup times for ride requests. The performance of graph operations depends primarily on the number of locations and roads contained within the network.

4.2 Module 2: Hash-Based Passenger and Driver Lookup

The Hash Table module stores passenger and driver information using open addressing with linear probing for collision handling.

Operation	Average Case	Worst Case	Space Complexity
Insert	$O(1)$	$O(N)$	$O(N)$
Search	$O(1)$	$O(N)$	$O(N)$
Delete	$O(1)$	$O(N)$	$O(N)$

Hash tables provide efficient storage and retrieval of passenger and driver records through key-based access. Under normal conditions, insertion, searching, and deletion operations execute in constant time. However, excessive collisions may cause linear probing to examine multiple locations, resulting in worst-case linear performance. The use of hashing significantly improves efficiency compared to sequential searching through records.

4.3 Module 3: Heap-Based Pickup Scheduling

The Heap module implements a max heap to manage ride requests according to their calculated priority values.

Operation	Time complexity	Space complexity
Insert a ride request	$O(\log N)$	$O(N)$
Delete the highest priority request	$O(\log N)$	$O(N)$
Peek the highest priority request	$O(1)$	$O(1)$

The heap structure maintains ride requests in priority order. When a new request is inserted, the heap performs a trickle-up operation to restore heap order. Similarly, removing the highest-priority ride requires a trickle-down operation. These operations execute in logarithmic time, allowing the system to efficiently manage large numbers of ride requests while ensuring that the most urgent requests are dispatched first.

4.4 Module 4: Sorting Pickup Requests

The Sorting module implements Merge Sort and Quick Sort to analyze sorting performance on ride request datasets.

Algorithm	Best case	Average case	Worst case	Space complexity
Merge sort	$O(N \log N)$	$O(N \log N)$	$O(N \log N)$	$O(N)$
Quick sort	$O(N \log N)$	$O(N \log N)$	$O(N^2)$	$O(N \log N)$

Merge Sort provides consistent performance across all input conditions because the dataset is repeatedly divided into smaller sub-arrays before being merged in sorted order. Quick Sort generally performs efficiently on average but may degrade to quadratic time when poor pivot selections occur. The sorting module compares both algorithms using datasets of varying sizes and arrangements to evaluate their performance characteristics and identify the most suitable approach for different

scenarios. Execution times were measured using Java's `System.nanoTime()` method. Nanosecond measurements were selected because the sorting operations execute very quickly on the tested datasets, making finer-grained timing more appropriate than milliseconds. The recorded execution times were used together with comparison counts to evaluate the relative performance of Merge Sort and Quick Sort.

5. Sample Outputs of Code

This section presents sample outputs obtained from testing each module of the ZipRide RideDispatch System. The screenshots demonstrate the successful implementation and operation of the graph, hash table, heap, and sorting modules. Each output verifies that the corresponding algorithms and data structures function according to the specified requirements.

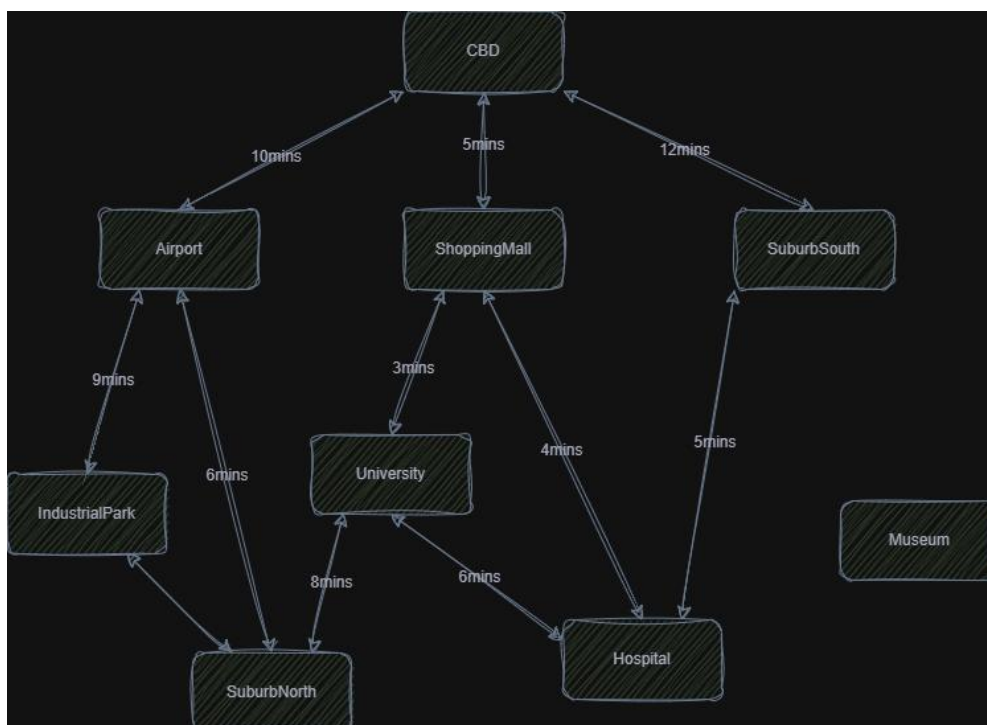


Figure 2 is the road network in a graph structure I have implemented

5.1 Module1:

5.1.1 Road network

```

=====
                        ROAD NETWORK
=====
1> Display Road Network
2> Display Locations
3> Add Location (Node)
4> Add Road (Edge)
5> Remove Location (Node)
6> Remove Road (Edge)
7> Breadth First Search (BFS)
8> Depth First Search (DFS)
9> Check for Cycles
10> Find Shortest Path
0> Back
=====
Enter choice: 1

Location: CBD
Airport -> 10 min
ShoppingMall -> 5 min
SuburbSouth -> 12 min

Location: Airport
CBD -> 10 min
SuburbNorth -> 6 min
IndustrialPark -> 9 min

Location: ShoppingMall
CBD -> 5 min
Hospital -> 4 min
University -> 3 min

Location: Hospital
ShoppingMall -> 4 min
University -> 6 min
SuburbSouth -> 5 min

Location: University
Hospital -> 6 min
SuburbNorth -> 8 min
ShoppingMall -> 3 min

Location: SuburbNorth
University -> 8 min
IndustrialPark -> 7 min
Airport -> 6 min

Location: IndustrialPark
SuburbNorth -> 7 min
Airport -> 9 min

Location: SuburbSouth
CBD -> 12 min
Hospital -> 5 min

Location: Museum
No roads

```

5. 1.2 Breadth First Search (BFS)

```
=====
ROAD NETWORK
=====
1> Display Road Network
2> Display Locations
3> Add Location (Node)
4> Add Road (Edge)
5> Remove Location (Node)
6> Remove Road (Edge)
7> Breadth First Search (BFS)
8> Depth First Search (DFS)
9> Check for Cycles
10> Find Shortest Path
0> Back
=====
Enter choice: 7
Enter starting location: CBD

BFS Traversal from: CBD

Level 0:
  CBD

Level 1:
  Airport
  ShoppingMall
  SuburbSouth

Level 2:
  SuburbNorth
  IndustrialPark
  Hospital
  University
```

5.1.3 Depth First Search

```
=====
ROAD NETWORK
=====
1> Display Road Network
2> Display Locations
3> Add Location (Node)
4> Add Road (Edge)
5> Remove Location (Node)
6> Remove Road (Edge)
7> Breadth First Search (BFS)
8> Depth First Search (DFS)
9> Check for Cycles
10> Find Shortest Path
0> Back
=====
Enter choice: 8
Enter starting location: CBD
DFS Traversal from: CBD
CBD, Airport, SuburbNorth, University, Hospital, ShoppingMall, SuburbSouth, IndustrialPark
```

5.1.4 Check for cycles

```
=====
ROAD NETWORK
=====
1> Display Road Network
2> Display Locations
3> Add Location (Node)
4> Add Road (Edge)
5> Remove Location (Node)
6> Remove Road (Edge)
7> Breadth First Search (BFS)
8> Depth First Search (DFS)
9> Check for Cycles
10> Find Shortest Path
0> Back
=====
Enter choice: 9

Cycle Detection: Detected!
Nodes involved: CBD -> Airport -> SuburbNorth -> University -> Hospital -> ShoppingMall -> CBD
```

5.1.5 Shortest Path

```

=====
                ROAD NETWORK
=====
1> Display Road Network
2> Display Locations
3> Add Location (Node)
4> Add Road (Edge)
5> Remove Location (Node)
6> Remove Road (Edge)
7> Breadth First Search (BFS)
8> Depth First Search (DFS)
9> Check for Cycles
10> Find Shortest Path
0> Back
=====
Enter choice: 10
Source location: IndustrialPark
Destination location: CBD

Dijkstra: IndustrialPark -> CBD
shortest path: IndustrialPark -> Airport -> CBD
Total driving time: 19 mins

```

5.2 Module2:

5.2.1 Add passenger

```

=====
                PASSENGER MANAGEMENT
=====
1> Display All Passengers
2> Search Passenger
3> Add Passenger
4> Delete Passenger
5> Update Membership Tier
6> Save passengers to CSV
0> Back
=====
Enter choice: 3
Passenger ID: 122
Passenger Name: Liyaana
Pickup Location: IndustrialPark
Membership Tier (1-5): 2
Passenger added successfully...

```

5.2.2 Search Passenger

```

=====
                PASSENGER MANAGEMENT
=====
1> Display All Passengers
2> Search Passenger
3> Add Passenger
4> Delete Passenger
5> Update Membership Tier
6> Save passengers to CSV
0> Back
=====
Enter choice: 2
Enter Passenger ID: 101
PassengerID: 101 | PassengerName: Isini | pickUp: CBD | MembershipTier: 1

```

5.2.3 Collision Handling

```

=====
          PASSENGER MANAGEMENT
=====
1> Display All Passengers
2> Search Passenger
3> Add Passenger
4> Delete Passenger
5> Update Membership Tier
6> Save passengers to CSV
0> Back
=====
Enter choice: 3
Passenger ID: 2048
Passenger Name: Ruhi
Pickup Location: Airport
Membership Tier (1-5): 2
Collision at index 27
Collision at index 28
Passenger added successfully...

```

5.2.4 Add driver

```

=====
          DRIVER MANAGEMENT
=====
1> Display All Drivers
2> Search Driver
3> Add Driver
4> Delete Driver
5> Update Driver Status
6> Save drivers to CSV
0> Back
=====
Enter choice: 3
Driver ID: 221
Driver Name: Riya
Current Location: CBD
Status (Available/Busy/Offline): Busy
Driver added successfully...

```

5.2.5 Update driver status

```

=====
          DRIVER MANAGEMENT
=====
1> Display All Drivers
2> Search Driver
3> Add Driver
4> Delete Driver
5> Update Driver Status
6> Save drivers to CSV
0> Back
=====
Enter choice: 5
Enter Driver ID: 221

Current Status: Busy

Available Status Values:
Available
Busy
Offline

Enter New Status: Offline
Driver status updated...

```

5.2.6 Delete driver

```
=====
DRIVER MANAGEMENT
=====
1> Display All Drivers
2> Search Driver
3> Add Driver
4> Delete Driver
5> Update Driver Status
6> Save drivers to CSV
0> Back
=====
Enter choice: 4
Enter Driver ID: 221
Driver deleted successfully...
```

5.3 Module3:

5.3.1 Create ride request

```
=====
RIDE SCHEDULING
=====
1> Create Ride Request
2> View Priority Queue
3> View Highest Priority Ride
4> Dispatch Next Ride
0> Back
=====
Enter choice: 1
Enter Passenger ID: 112

Assigned Driver: Shalindra
Pickup Time: 1 mins
Priority = (6-2)+ 1000/1 = 1004.0

Inserting...
Methuki (Priority: 1004.0)
Heap state [1 requests]:
[0] Methuki | Tier: 2 | Time: 1 | Priority: 1004.00
```

5.3.2 View priority queue

```
=====
RIDE SCHEDULING
=====
1> Create Ride Request
2> View Priority Queue
3> View Highest Priority Ride
4> Dispatch Next Ride
0> Back
=====
Enter choice: 2
Heap state [3 requests]:
[0] Isini | Tier: 1 | Time: 1 | Priority: 1005.00
[1] Methuki | Tier: 2 | Time: 1 | Priority: 1004.00
[2] Pasandul | Tier: 5 | Time: 1 | Priority: 1001.00
```

5.3.3 View highest priority

```

=====
RIDE SCHEDULING
=====
1> Create Ride Request
2> View Priority Queue
3> View Highest Priority Ride
4> Dispatch Next Ride
0> Back
=====
Enter choice: 3
PassengerID: 101 | Name: Isini | Tier: 1 | Pickup Time: 1 mins | Driver: Raed (217) | Priority: 1005.00

```

5.3.4 Dispatch next ride

```

=====
RIDE SCHEDULING
=====
1> Create Ride Request
2> View Priority Queue
3> View Highest Priority Ride
4> Dispatch Next Ride
0> Back
=====
Enter choice: 4

[EXTRACT PRIORITY] Dispatching: Isini (Priority: 1005.0) | Driver: Raed
Heap state [2 requests]:
[0] Methuki | Tier: 2 | Time: 1 | Priority: 1004.00
[1] Pasandul | Tier: 5 | Time: 1 | Priority: 1001.00

```

5.4 Module4:

5.4.1 Merge sort execution

```

=====
SORTING & ANALYSIS
=====
1> Run Merge Sort
2> Run Quick Sort
3> Compare Merge Sort and Quick Sort
0> Back
=====
Enter choice: 1

Merge Sort Completed...
Comparisons: 504
Sorted correctly: true
First 5 records:
[0] PickupTime=1 mins | Isini
[1] PickupTime=1 mins | Mario
[2] PickupTime=1 mins | Liyaana
[3] PickupTime=1 mins | Isini
[4] PickupTime=1 mins | Mario
Last 5 records:
[95] PickupTime=11 mins | Samadhi
[96] PickupTime=11 mins | Pasandul
[97] PickupTime=11 mins | Samadhi
[98] PickupTime=11 mins | Pasandul
[99] PickupTime=11 mins | Samadhi

```

5.4.2 Quick sort execution

```

=====
      SORTING & ANALYSIS
=====
1> Run Merge Sort
2> Run Quick Sort
3> Compare Merge Sort and Quick Sort
0> Back
=====
Enter choice: 2

Quick Sort Completed...
Comparisons: 602
Sorted correctly: true
First 5 records:
  [0] PickupTime=1 mins | Isini
  [1] PickupTime=1 mins | Mario
  [2] PickupTime=1 mins | Mario
  [3] PickupTime=1 mins | Liyaana
  [4] PickupTime=1 mins | Isini
Last 5 records:
  [95] PickupTime=11 mins | Samadhi
  [96] PickupTime=11 mins | Pasandul
  [97] PickupTime=11 mins | Pasandul
  [98] PickupTime=11 mins | Samadhi
  [99] PickupTime=11 mins | Pasandul

```

5.4.3 Comparison of Merge and Quick sort

```

=====
      SORTING & ANALYSIS
=====
1> Run Merge Sort
2> Run Quick Sort
3> Compare Merge Sort and Quick Sort
0> Back
=====
Enter choice: 3

--- Size: 100 ---
RANDOM | Merge: 95366ns (519 comps) | Quick: 6425582ns (614 comps)
NEARLY_SORTED | Merge: 11415ns (460 comps) | Quick: 49618ns (616 comps)
REVERSED | Merge: 8494ns (428 comps) | Quick: 19592ns (607 comps)

--- Size: 500 ---
RANDOM | Merge: 51603ns (3537 comps) | Quick: 120872ns (4331 comps)
NEARLY_SORTED | Merge: 46351ns (3240 comps) | Quick: 118672ns (4238 comps)
REVERSED | Merge: 169594ns (2690 comps) | Quick: 272580ns (4285 comps)

--- Size: 1000 ---
RANDOM | Merge: 264364ns (8014 comps) | Quick: 122471ns (9160 comps)
NEARLY_SORTED | Merge: 112441ns (7183 comps) | Quick: 50408ns (10043 comps)
REVERSED | Merge: 91927ns (5294 comps) | Quick: 110965ns (9624 comps)

```

6. Module 4: Discussion

6.1 Merge Sort

Merge Sort is a stable sorting algorithm. This means that if two ride requests have the same pickup time, their original relative order is preserved after sorting. Stability can be useful when records contain additional information that should maintain its original sequence when equal sorting keys are encountered. The Merge Sort implementation in the Zip ride system maintains this stability during the merge operation.

6.2 Quick Sort

The Quick Sort implementation uses a median-of-three pivot selection strategy. The pivot is chosen as the median of the first, middle, and last elements of the current subarray. This approach helps avoid poor pivot selections that can occur when the input data is already sorted or reverse sorted. By producing more balanced partitions, the median-of-three strategy generally improves sorting efficiency and reduces the likelihood of worst-case performance.

6.3 Comparison

Size= 100		
Random	Merge: 119300ns (546 comps)	Quick: 48800ns (608 comps)
Nearly Sorted	Merge: 16400ns (477 comps)	Quick: 21300ns (624 comps)
Reversed	Merge: 12100ns (392 comps)	Quick: 29000ns (628 comps)

Size= 500		
Random	Merge: 58700ns (3842 comps)	Quick: 238900ns (4352 comps)
Nearly Sorted	Merge: 52000ns (3487 comps)	Quick: 61900ns (4361 comps)
Reversed	Merge: 41200ns (2620 comps)	Quick: 19400ns (4341 comps)

Size= 1000		
Random	Merge: 173300ns (8634 comps)	Quick: 66900ns (10722 comps)
Nearly Sorted	Merge: 87900ns (7888 comps)	Quick: 43200ns (9850 comps):
Reversed	Merge: 90400ns (5998 comps)	Quick: 45100ns (9755 comps)

7. Conclusion

The Zip ride Ride-Dispatch System successfully integrates graphs, hash tables, heaps, and sorting algorithms to support route management, record storage, ride scheduling, and performance analysis. Testing confirmed that all modules operate according to the specified requirements. The Graph module accurately performs traversal and shortest-path calculations, the Hash Table module efficiently manages passenger and driver records, the Heap module prioritises ride requests correctly, and the Sorting module successfully compares Merge Sort and Quick Sort under different dataset conditions. Overall, the project demonstrates the practical application of fundamental data structures and algorithms in solving real-world ride-dispatch problems.

8. References

- GeeksforGeeks. (2025). *Java Program to Detect Cycle in a Directed Graph*. Retrieved from GeeksforGeeks: <https://www.geeksforgeeks.org/java/java-program-for-detect-cycle-in-undirected-graph/>
- GeeksforGeeks. (2026). *Breadth First Search or BFS for a Graph*. Retrieved from GeeksforGeeks: <https://www.geeksforgeeks.org/dsa/breadth-first-search-or-bfs-for-a-graph/>
- GeeksforGeeks. (2026). *Depth First Search or DFS for a Graph*. Retrieved from GeeksforGeeks: <https://www.geeksforgeeks.org/dsa/depth-first-search-or-dfs-for-a-graph/>
- GeeksforGeeks. (2026). *Dijkstra's Algorithm*. Retrieved from GeeksforGeeks: <https://www.geeksforgeeks.org/dsa/dijkstras-shortest-path-algorithm-greedy-algo-7/>
- W3Schools. (2026). *DSA Dijkstra's Algorithm*. Retrieved from W3Schools: https://www.w3schools.com/dsa/dsa_algo_graphs_dijkstra.php
- W3Schools. (2026). *DSA Graphs Cycle Detection*. Retrieved from W3Schools: https://www.w3schools.com/dsa/dsa_algo_graphs_cycledetection.php